

Kalshi Crypto Trading Bot — Technical Whitepaper

Gabriel Kagan

February 2026

Contents

Part 1: Executive Summary	2
What It Does	2
Market Opportunity	2
Strategy in Plain English	3
Key Differentiators	3
Part 2: System Architecture	3
Data Pipeline	3
Component Overview	4
Data Sources	5
Part 3: Technical Deep-Dive	5
3.1 Volatility Engine	5
Realized Kernel (Barndorff-Nielsen 2008)	5
Time-Varying RK Weights	6
Mincer-Zarnowitz R^2 -Weighted Blending	6
EGARCH(1,1) with Student-t Innovations	6
Adaptive Jump Detection	7
DVOL Integration	7
Cross-Asset Beta	7
3.2 Probability Model	7
Step 1: Z-Score Computation	7
Step 2: NIG CDF (Per-Asset Fitted)	7
Step 3: Data-Driven Calibration	8
Step 4: Dynamic Probability Cap	8
Step 5: Market-Price Blending	8
Sanity Checks	9
3.3 Edge Detection	9
Fee Formula	9
Fee-Adjusted Edge	9
3.4 Kalshi Order Flow (Shadow Mode)	10
3.5 Execution Strategy	10
Three-Tier Post-Only Handler	10
Time-Based Escalation	10
Maker-Only Threshold	10

Maker-to-Taker Conversion	11
Partial Fill Handling	11
UUID Persistence	11
3.6 Position Sizing	11
Edge-Tiered Sizing	11
Drawdown Scaling	11
Part 4: Risk Management	12
Position Sizing Controls	12
Market Selection Controls	12
Model Sanity Controls	12
Execution Controls	12
Part 5: Performance	13
Live Trading Results	13
Markets	13
15-Minute Markets (Live Trading)	13
Hourly Markets (Observation Mode)	13
Part 6: Infrastructure	13
Deployment	13
Data Persistence	14
SQLite (state.db)	14
JSONL Journals	14
Firebase Real-Time Dashboard	14
Shadow Mode Features	14

Part 1: Executive Summary

What It Does

This system is an automated trading bot for **Kalshi**, a CFTC-regulated prediction market exchange. It trades **15-minute cryptocurrency price threshold contracts** — binary options that pay \$1 if a crypto asset (BTC, ETH, SOL, or XRP) stays above a given price at the end of a 15-minute window, and \$0 otherwise.

The bot monitors real-time prices across multiple exchanges, estimates the probability of each outcome using microstructure-aware volatility models with EGARCH conditioning, and places trades when its model identifies a statistical edge over the market price.

Market Opportunity

Kalshi lists 15-minute crypto contracts around the clock. Each window produces fresh contracts for four assets at multiple strike prices, creating hundreds of tradeable markets per day. Because these are short-duration, binary-outcome instruments, mispricing tends to be small but frequent — an ideal environment for systematic, model-driven trading.

The bot also monitors hourly crypto markets (75 strikes per event) in observation mode, collecting calibration data for future live trading.

Strategy in Plain English

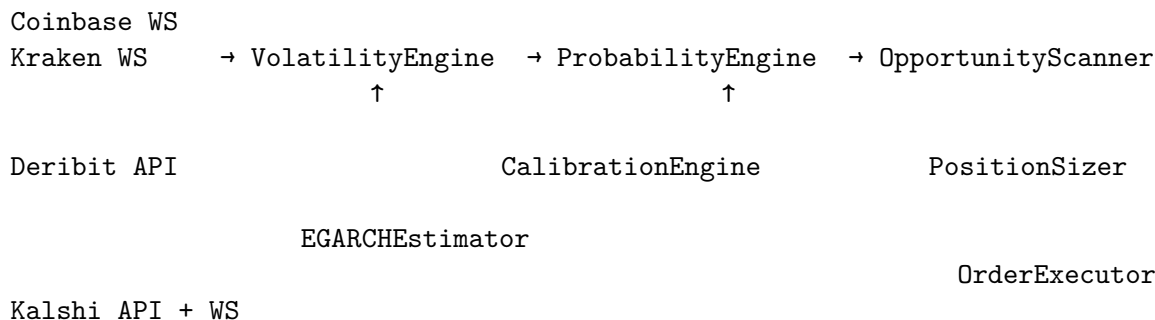
1. **Observe** — Continuously stream spot prices from Coinbase and Kraken. Fetch implied volatility from Deribit. Monitor Kalshi's own orderbook via WebSocket.
2. **Estimate** — For every active market, compute the probability that the asset stays above its threshold using EGARCH-conditioned volatility with fat-tailed NIG distributions fitted per asset.
3. **Filter** — Reject markets that are too uncertain, too expensive, or offer insufficient edge after fees.
4. **Size** — Use edge-tiered position sizing with automatic drawdown scaling.
5. **Execute** — Place maker (limit) orders first to minimize fees, with three-tier post_only rejection handling and time-aware taker escalation. No taker execution below 90 seconds to close.
6. **Settle** — Track outcomes via the Kalshi settlements API and log performance for continuous evaluation.

Key Differentiators

- **Multi-exchange intelligence:** Aggregates spot prices from Coinbase and Kraken plus derivatives signals from Deribit, detecting cross-exchange lead-lag patterns before they appear in Kalshi prices.
- **EGARCH-conditioned volatility:** Realized Kernel estimation (Barndorff-Nielsen 2008) with data-adaptive bandwidth, Mincer-Zarnowitz R^2 -weighted blending, and EGARCH(1,1) conditional volatility — all promoted to live trading.
- **Per-asset NIG distributions:** Normal Inverse Gaussian CDF replaces the generic Student-t, capturing both heavy tails and asymmetry specific to each cryptocurrency.
- **Adaptive execution:** Three-tier post_only rejection handler (normal → degraded → taker IOC), maker-first strategy with time-aware escalation, and maker-only threshold below 90 seconds.
- **Data-driven risk controls:** Edge-tiered sizing (25% max), drawdown scaling, z-score sanity checks, data-driven calibration, and model-market discrepancy detection.

Part 2: System Architecture

Data Pipeline



SettlementTracker → StateManager (SQLite) Logger (JSONL journals)

Component Overview

Component	Role
KalshiClient	API communication with RSA-PSS authentication and per-second rate limiting (30 reads/sec, 30 writes/sec on Advanced tier)
KalshiFeed	WebSocket connection for real-time fills and orderbook delta streaming
Logger	Structured JSONL logging across multiple journals with fill deduplication
StateManager	SQLite-backed persistent state (WAL mode for crash resilience); tracks positions, orders, fills, and settlements
CoinbaseFeed	Real-time WebSocket feed for BTC, ETH, SOL, XRP with 10,800-point price buffer (15 hours at 5-second intervals)
DeribitDVOLFetcher	Daemon thread fetching implied volatility (DVOL) index for BTC and ETH every 60 seconds
CrossExchangeFeed	WebSocket feed from Kraken for cross-exchange lead-lag detection
KalshiOrderFlowTracker	Shadow-mode Kalshi-native orderbook imbalance, depth velocity, and spread convergence signals
VolatilityEngine	Realized Kernel volatility with adaptive bandwidth (H^*), MZ R^2 -weighted blending, EGARCH(1,1) Student-t conditioning, time-varying RK weights, adaptive jump detection, and DVOL integration
EGARCHEstimator	EGARCH(1,1) with Student-t innovations (df 3.2–3.8), MLE-fitted on 10,800 samples (15 hours), refitted hourly — live (promoted from shadow)
MZTracker	Mincer-Zarnowitz R^2 regression for dynamic EGARCH blend weight estimation with EMA smoothing
ProbabilityEngine	Win probability via NIG CDF (per-asset fitted) with data-driven calibration, dynamic caps, and market-price blending
CalibrationEngine	Learns calibration from settlement outcomes: Platt Scaling → Beta Calibration (currently active with 1,900+ observations)
PositionSizer	Edge-tiered position sizing with drawdown-based scaling

Component	Role
OpportunityScanner	Multi-stage filter pipeline evaluating all markets across active 15-minute windows and hourly observation windows
OrderExecutor	Three-tier post_only handler, maker-first limit orders with adaptive taker escalation, WebSocket fill detection, amend-first conversion, maker-only below 90s
SettlementTracker	Incremental settlement polling (30-second intervals) using the Kalshi settlements API
TelegramNotifier	Optional Telegram alerts for trades, settlements, and errors
MainLoop	Continuous 1-second observation loop coordinating all components

Data Sources

Source	Data	Transport	Frequency
Coinbase	Spot prices (BTC, ETH, SOL, XRP)	WebSocket	Real-time (5s snapshots, 10,800-point buffer = 15hr)
Kraken	Spot prices (cross-exchange)	WebSocket	Real-time
Deribit	Implied volatility (DVOL) for BTC/ETH	REST API	60 seconds
Kalshi	Markets, orderbooks, balance, fills, settlements	REST API + WebSocket	On-demand + real-time
Binance	Spot prices (cross-exchange)	WebSocket	Geo-blocked (HTTP 451 on VPS)

Part 3: Technical Deep-Dive

3.1 Volatility Engine

The volatility engine produces a per-asset, per-5-second realized volatility estimate that feeds the probability model. It combines multiple techniques with data-adaptive weighting and EGARCH forward-looking conditioning.

Realized Kernel (Barndorff-Nielsen 2008)

Standard sample variance of high-frequency returns is biased by market microstructure noise (bid-ask bounce, discrete tick sizes). The Realized Kernel estimator corrects this using a kernel-weighted

autocovariance function with a Parzen flat-top kernel, producing noise-robust volatility estimates from 5-second log returns.

****Adaptive bandwidth (H*)**:** Rather than using a fixed bandwidth $H=1$, the system estimates the optimal bandwidth from the data using the noise-to-signal ratio:

$$H^* = c \times \left(\frac{\hat{\omega}^2}{IV} \right)^{2/5} \times n^{3/5}$$

where $\hat{\omega}^2$ is the estimated microstructure noise variance and IV is the integrated variance. This produces tighter estimates during calm periods and wider smoothing during noisy periods.

Time-Varying RK Weights

Multiple RK estimators at different scales are blended using time-varying weights that adapt to current market conditions, rather than fixed proportions.

Mincer-Zarnowitz R²-Weighted Blending

Rather than fixed weights, the system dynamically weights estimators based on their forecasting quality. A Mincer-Zarnowitz regression compares each estimator's forecast against realized outcomes:

$$RV_{t+1} = \alpha + \beta \times \hat{\sigma}_t + \varepsilon_t$$

The R^2 from this regression measures forecast quality. Weights are smoothed using an EMA ($=0.97$) to prevent whipsawing:

$$w_t = \lambda \times w_{t-1} + (1 - \lambda) \times w_{raw}$$

Below an R^2 threshold of 0.10, the system reverts to equal-weight blending as a fallback.

EGARCH(1,1) with Student-t Innovations

The EGARCH model captures volatility clustering and leverage effects:

$$\log(\sigma_t^2) = \omega + \alpha (|z_{t-1}| - E[|z|]) + \gamma z_{t-1} + \beta \log(\sigma_{t-1}^2)$$

Fitted with Student-t innovations (df typically 3.2–3.8 for crypto) via maximum likelihood on 10,800 samples (15 hours at 5-second intervals), refitted hourly. The EGARCH forecast is blended with the RK estimate using MZ R^2 -weighted blending — the system dynamically determines how much weight to give the conditional model vs. the realized estimate based on forecast quality.

Status: Live — promoted from shadow mode after extensive validation. EGARCH core vol, EGARCH blend, and MZ R^2 -weighted blending all drive live trading decisions.

Adaptive Jump Detection

Jumps — sudden, large price moves — invalidate smooth volatility assumptions. The system uses an adaptive threshold rather than the fixed 3 σ approach:

- **EWMA variance tracking:** Tracks running variance of 15-second subsampled returns ($\lambda = 0.94$)
- **Percentile-based threshold:** Jump trigger set at the asset’s own volatility distribution percentile, adapting to current regime
- **Tiered response:** Jump multiplier and cooldown duration scale with jump severity
- **Health monitoring:** Logs EWMA σ , percentile, threshold, and total jump count per asset

DVOL Integration

When Deribit implied volatility (DVOL) exceeds realized volatility by more than 50%, the system blends in the implied estimate using inverse-variance weighting. This respects the market’s forward-looking information during regime changes while anchoring to observed data.

Cross-Asset Beta

For assets without direct DVOL data (SOL, XRP), the system estimates a cross-asset beta against BTC using a 60-return lookback window, clamped to $[0.5, 3.0]$, allowing derivative signals to propagate across correlated assets.

3.2 Probability Model

Given the blended volatility, the probability engine estimates the likelihood that an asset stays above its threshold for the remaining window duration.

Step 1: Z-Score Computation

$$z = \frac{\text{threshold} - \text{spot}}{\text{spot} \times \sigma_{\text{blended}} \times \sqrt{T/5}}$$

where T is seconds remaining and σ_{blended} is per-5-second scale. The denominator represents the expected magnitude of price movement over the remaining window.

Step 2: NIG CDF (Per-Asset Fitted)

The model uses the Normal Inverse Gaussian distribution with per-asset fitted parameters:

$$p_{\text{raw}} = 1 - F_{\text{NIG}}(z; a, b, \mu, \delta)$$

where a controls tail heaviness, b captures asymmetry (skew), μ is location, and δ is scale. NIG parameters are fitted via maximum likelihood on 7 days of 60-second returns (~10,000 samples per asset) and stored in `dist_config.json`.

Why NIG over Student-t? NIG provides two key improvements: - **Asymmetry:** The b parameter captures the empirical skew in crypto returns (e.g., BTC $b = -0.019$, slight left skew) - **Better tail fit:** KS test p-values for NIG are dramatically higher (BTC: 0.11, ETH: 0.42) compared to Student-t (effectively 0), indicating NIG genuinely captures the return distribution

The system falls back to Student-t(df=4) if NIG parameters are unavailable.

Step 3: Data-Driven Calibration

Raw probabilities are calibrated using a CalibrationEngine that learns from settlement outcomes:

Method	Min Samples	Description
Fixed logistic ($\alpha=0.85$)	0	Default fallback — compresses extreme probabilities
Platt Scaling	200	2-parameter logistic (A, B) fitted to outcomes
Beta Calibration	350	3-parameter (a, b, c) — more flexible than Platt

The engine automatically promotes to better methods as data accumulates, with validation checks to prevent degradation. Currently running Beta Calibration with 1,900+ observations.

Step 4: Dynamic Probability Cap

A time-dependent cap adjusts confidence based on time remaining:

Time to Expiry	Cap
> 10 minutes	93%
5–10 minutes	95%
2–5 minutes	97%
1–2 minutes	98.5%
< 1 minute	99.5%

As expiry approaches and less can go wrong, the cap relaxes to allow higher-confidence trades in the endgame.

Step 5: Market-Price Blending

The calibrated probability is blended with the market-implied probability:

$$p_{final} = 0.60 \times p_{cal} + 0.40 \times p_{market}$$

This 60/40 blend (60% model, 40% market) was validated against a no-blend alternative: the no-blend system was +1.86 percentage points overconfident (Brier score 0.0946 vs 0.0422), and would have generated 16 trades that were net -\$53.54. The 40% market weight was subsequently tuned from 50% after data showed the model was underconfident by 0.8–2.1pp at 90%+ probabilities. The no-blend system now monitors in shadow mode.

Sanity Checks

- **Z-score limit:** If $|z| > 25$, the market is refused (all historical losses have z-scores below this threshold; the previous limit of 12 was blocking only winners)
- **Model-market discrepancy:** If $p_{cal} > 90\%$ but market price $< 75\text{¢}$, the market is refused (the model may be missing information the market has)

3.3 Edge Detection

A trade requires positive expected value after accounting for fees.

Fee Formula

Kalshi charges fees using a variance-based formula:

$$\begin{aligned}\text{taker fee} &= \lceil 0.07 \times C \times P \times (1 - P) \rceil \text{ cents} \\ \text{maker fee} &= \lceil 0.0175 \times C \times P \times (1 - P) \rceil \text{ cents}\end{aligned}$$

where C is the number of contracts and P is the trade price as a decimal. The ceiling is applied to the total, not per contract.

Fee-Adjusted Edge

The scanner evaluates edge using taker fees (worst-case), so any candidate that passes the filter is profitable even if maker order is rejected:

$$\text{edge} = p_{final} - \frac{\text{best_ask}}{100} - \frac{\text{taker_fee}}{C \times 100}$$

A trade must satisfy:

$$\text{edge} \geq \text{get_min_edge}(\text{price})$$

The minimum edge is price-dependent, reflecting the higher risk of expensive contracts:

Entry Price	Min Edge
97¢+	4.0%
95–96¢	2.5%
93–94¢	1.8%
91–92¢	1.2%
89–90¢	0.9%
87–88¢	0.7%

A flat fallback of 0.7% (MIN_EDGE_PCT) applies if the price-dependent schedule is unavailable.

3.4 Kalshi Order Flow (Shadow Mode)

The KalshiOrderFlowTracker monitors Kalshi's own orderbook for predictive signals:

- **Imbalance:** Ratio of bid vs. ask depth — strong imbalance (>0.8 or <0.2) suggests directional pressure
- **Depth velocity:** Rate of change in total depth — draining liquidity may predict a move
- **Spread convergence:** Narrowing spread + trending depth suggests informed trading
- **Adjustments:** ± 1 to 1.5pp based on signal strength, with confidence levels based on snapshot count

Currently logging only — signals are computed but do not affect trading decisions.

3.5 Execution Strategy

The executor uses a maker-first approach with three-tier post_only rejection handling, time-aware escalation, and a hard maker-only threshold.

Three-Tier Post-Only Handler

When a `post_only=True` maker order is rejected (the order would cross the spread rather than rest on the book), the system escalates through three tiers:

Tier	Trigger	Action
Tier 1: Normal maker	0–1 rejections	Standard maker order, 1–2¢ below fair value
Tier 2: Degraded maker	2 rejections	Same offset + 1¢ additional discount. If price drops below 87¢ floor, skipped.
Tier 3: Taker IOC	3+ rejections	Edge re-verified with actual taker fees → IOC order if still profitable

Rejection counts expire after 30 seconds and are per-ticker (unique per market window).

Time-Based Escalation

For orders that are successfully placed but sit unfilled:

Urgency	Time to Close	Maker Wait
Low	90–270s	15s
Medium	60–90s	10s
High	30–60s	5s

Maker-Only Threshold

No taker execution below 90 seconds to close. All taker paths — direct taker, post-only taker escalation, early/standard escalation — are blocked. Maker orders are still submitted and can fill. Data: taker trades below 90 seconds cost -\$85 in net losses.

Maker-to-Taker Conversion

1. Place maker order with `post_only=True` (guarantees maker fees, 4× cheaper)
2. Monitor for fills via Kalshi WebSocket (zero API cost) with REST polling fallback
3. Poll queue position every ~5s for queue-aware escalation timing
4. If timeout reached without fill (and above 90s to close):
 - Attempt `amend_order()` to convert to taker price in-place (avoids cancel+replace race)
 - If amend fails, fall back to cancel + IOC (`time_in_force="immediate_or_cancel"`) taker order
 - Re-validate price still in [87¢, 99¢] before taker submission

Partial Fill Handling

Orders may partially fill (e.g., 3 of 13 contracts). The execution engine tracks `filled_so_far` cumulatively and keeps the order active until fully filled or escalated. REST fill detection uses a `_seen_fill_ids` set to prevent double-counting across consecutive polls and against WebSocket fills. Escalation to taker subtracts partial fills from the IOC count to prevent position doubling.

UUID Persistence

Each order gets a `client_order_id` (UUID4) written to SQLite before API submission. This ensures crash recovery — if the bot restarts mid-order, it can reconcile using the persisted UUID.

3.6 Position Sizing

Edge-Tiered Sizing

Position sizing is tiered by fee-adjusted edge, with higher-conviction trades receiving larger allocations:

Fee-Adjusted Edge	Risk Fraction
4%	25% of bankroll
2.5%	20% of bankroll
1.8%	15% of bankroll
1.2%	10% of bankroll
0.9%	7% of bankroll
0.7%	5% of bankroll

Safety ceiling: max 25% of bankroll at risk per trade.

Drawdown Scaling

Balance vs. Starting	Sizing Adjustment
85%	Full sizing
75–85%	Half sizing
65–75%	Quarter sizing
< 65%	Halt trading

This creates a geometric de-risking curve that preserves capital during losing streaks.

Part 4: Risk Management

Position Sizing Controls

- **Edge-tiered sizing:** Position size scales with conviction — 25% max at 4%+ edge, down to 5% at 0.7% edge
- **Drawdown scaling:** Size halved below 85% of starting balance, quartered below 75%, trading halted below 65%
- **Hard limits:** Maximum risk per trade capped at 25% of bankroll

Market Selection Controls

- **Multi-asset capable:** Can trade multiple assets per 15-minute window
- **Price range guardrails:** Only trade contracts priced 87–99¢ — below 87¢ has historically poor win rates; above 99¢ offers insufficient reward
- **Price-dependent edge threshold:** Fee-adjusted edge must exceed a price-dependent minimum (0.7% at 87¢ up to 4.0% at 97¢+) after taker fees (worst-case)
- **Scanner uses taker fees:** Every candidate is profitable even if forced to taker execution

Model Sanity Controls

- **Z-score limit:** Refuse markets where $|z| > 25$ (validated against settlement data: 82 tradeable z-score rejections above 12 were all winners, leading to the raise from 12 $\rightarrow$ 25)
- **Model-market discrepancy:** If the model estimates >90% probability but the market prices below 75¢, refuse (the model may be missing material information)
- **Dynamic probability cap:** Time-dependent ceiling (93–99.5%) prevents overconfidence regardless of model output
- **Data-driven calibration:** CalibrationEngine learns from settlement outcomes, replacing fixed assumptions with empirical mappings
- **Market-price blending:** 60/40 blend (60% model, 40% market) anchors estimates and prevents systematic overconfidence

Execution Controls

- **Three-tier post_only handler:** Escalates from normal maker $\rightarrow$ degraded maker $\rightarrow$ taker IOC after repeated rejections, with edge re-verification at each tier
- **Maker-first with post_only:** Guarantees maker fee tier (75% cheaper), rejected if it would cross the spread
- **Maker-only below 90 seconds:** All taker execution paths blocked below 90s to close (data-driven: taker <90s cost -\$85)
- **WebSocket fill detection:** Zero-cost fill monitoring via Kalshi WebSocket, with REST polling fallback
- **Amend-first escalation:** Uses `amend_order()` API to convert maker $\rightarrow$ taker in-place, avoiding cancel+replace race conditions

- **IOC taker orders:** Taker escalation uses `time_in_force="immediate_or_cancel"` to prevent stale resting orders
- **Partial fill tracking:** Escalation subtracts filled contracts to prevent position doubling
- **Price re-validation:** After maker timeout, the system re-fetches the orderbook and re-validates the price range before submitting a taker order
- **UUID persistence:** Order IDs written to disk before API submission, enabling crash recovery without duplicate orders
- **Rejection expiry:** `Post_only` rejection counts expire after 30 seconds, preventing stale state from affecting future windows

Part 5: Performance

Live Trading Results

Metric	Value
Status	Live trading since February 22, 2026
Settled trades	0
Win rate	N/A (0W / 0L)
Assets	BTC, ETH, SOL, XRP

Markets

15-Minute Markets (Live Trading)

Binary contracts settling every 15 minutes. Series: KXBTC15M, KXETH15M, KXSOL15M, KXXRP15M.

Hourly Markets (Observation Mode)

75 strikes per event, settling every hour. Currently collecting calibration data only — no live trading. Series: KXBTCD, KXETHD, KXSOLD, KXXRPD.

Part 6: Infrastructure

Deployment

- **Host:** DigitalOcean droplet (Ubuntu 24.04)
- **Runtime:** Python 3, `virtualenv`
- **Process manager:** `systemd` (`kalshi-bot` service)
- **Startup:** `start.sh` activates `venv`, sources `.env`, launches `bot.py`
- **Auto-deploy:** Pushing to `main` triggers a GitHub Action that SSHes into the VPS, pulls the latest code, syntax-checks `bot.py`, and restarts the service

Data Persistence

SQLite (state.db)

The primary state store uses SQLite in WAL (Write-Ahead Logging) mode for crash resilience:

Table	Purpose
<code>positions</code>	Active positions (ticker, asset, side, count, avg price)
<code>pending_orders</code>	Orders awaiting fill (with UUID <code>client_order_id</code>)
<code>settled_trades</code>	Completed trades with P&L
<code>rejected_opportunities</code>	Markets rejected with reason and model state
<code>evaluated_opportunities</code>	Every market evaluation with filter stage (UNIQUE on ticker + stage)

JSONL Journals

Append-only journal files provide a complete audit trail:

Journal	Contents
<code>opportunity_journal.jsonl</code>	Filter stage tracking for every market evaluation
<code>scan_journal.jsonl</code>	Per-tick scan summaries (~330MB/day)
<code>rejection_journal.jsonl</code>	Settlement outcomes for rejected opportunities
<code>fill_model_journal.jsonl</code>	Maker order lifecycle data for ML fill prediction

Firebase Real-Time Dashboard

A Firebase integration provides a live web dashboard showing:

- Current positions and P&L
- Active market evaluations
- Volatility regime indicators and EGARCH/NIG parameters
- Orderbook visibility for active windows
- Execution engine statistics (amend success rate, WS fill ratio, post_only rejection counts, taker escalation counts)
- Calibration diagnostics
- Hourly observation stats

Shadow Mode Features

The system supports shadow mode for experimental features — they compute and log but do not affect live trading decisions:

Feature	Status	Purpose
Kalshi Order Flow	Shadow	Orderbook imbalance, depth velocity, spread convergence signals
Sigmoid QLIKE	Shadow	Alternative EGARCH weight via QLIKE improvement ratio
Shadow Cal Pipeline	Shadow	No-blend calibration monitoring (was promoted, caused +1.86pp overconfidence)
Dip Addon	Shadow	Buy more when ask dips 3¢ below entry after fill
Hourly Observation	Observation	Collecting calibration data for hourly markets (75 strikes/event)

Promoted features (driving live behavior): - **EGARCH core vol** — EGARCH(1,1) with Student-t innovations - **EGARCH blend** — MZ R^2 -weighted blending of EGARCH vs RK - **Time-varying RK weights** — adaptive multi-scale RK blending - **Adaptive jump detection** — percentile-based thresholds per asset - **Adaptive RK bandwidth** — data-driven H^* selection - **Temperature calibration** — competes in hourly Brier tournament

Last updated: N/A